

Setting up your computer

NSDF-Data student guide 1 — macOS edition

Version 11 · 9 October 2026

Tip

Who this is for. You are starting from little or no computing experience. You will paste some commands into a terminal and read the answers. You do **not** need to know any programming. Work through the steps in order. Each step ends with a **Checkpoint**: if it does not match, stop there.

Before you start

- **Time:** about one hour. Most of it is waiting for downloads and installers.
- **You need:** an internet connection, the password for your computer, and about 3 GB of free disk space.
- **Rule of thumb:** type or paste commands **exactly** as written. Capitals and spaces matter.
- **Nothing to be afraid of:** the commands in this guide install software and make new folders. None of them deletes your files. If you are not sure what a line does, there is a short table after each group of commands.
- **When something goes wrong:** do not guess. Copy the message, note the step you were on, and ask the project lead. See *If something goes wrong* at the end.

Words you will meet

Word	What it means
Terminal	A window where you type commands and the computer answers in text.
Command	One line you type in the terminal, followed by Enter (or Return).
Folder (or directory)	A place on your computer that holds files and other folders.
Path	The address of a folder, such as C:\Users\YourName\Research.
Git	A program that downloads and keeps track of versions of code.
Repository (repo)	A project folder that Git tracks. Here, NSDF-Data.

Word	What it means
Python	The programming language the analysis is written in. You will not need to write Python to finish these guides.
Conda / Miniforge	A tool that installs Python and packages in separate, named <i>environments</i> .
Environment	A sealed set of programs and versions, so this project's software does not clash with others.
Package	A piece of ready-made software (for example <i>numpy</i>).
Jupyter notebook	A document that mixes text, code and plots, run one piece (<i>cell</i>) at a time.
Branch	A separate line of work in Git, so your changes do not touch the main code.

Step 1. Open a terminal

Press **Command + Space**, type **Terminal**, and press **Return**. A window with a text prompt opens. This is the terminal for everything in this guide.

Paste with **Command + V**.

Checkpoint

You see a line of text ending in a symbol such as `>`, `$` or `%`. Type `echo hello` and press **Enter**: the word `hello` should appear under it.

Step 2. Install Git

Type this in the Terminal and press **Return**:

```
xcode-select --install
```

A pop-up appears. Click **Install**, agree to the licence, and wait. It can take 15 minutes or more. If the message says the tools are already installed, that is fine.

Checkpoint

In the terminal, type `git --version` and press **Enter**. You should see a line starting with `git version`.

Step 3. Install Miniforge

Miniforge installs Python and the other packages the analysis needs. Its download file is **Miniforge3-MacOSX-arm64.sh (Apple chip)** or **Miniforge3-MacOSX-x86_64.sh (Intel chip)**.

First find out which chip your Mac has: click the Apple menu, then **About This Mac**. If it says **Chip: Apple M** something, use the **arm64** file. If it says **Processor: Intel**, use the **x86_64** file.

Download the file into your **Downloads** folder: [Miniforge3-MacOSX-arm64.sh](#) (Apple chip) or

[Miniforge3-MacOSX-x86_64.sh](#) (Intel chip). If your browser asks whether to keep the file, choose **Keep**. Then, in the Terminal, run the command below (use the file name you downloaded):

```
bash ~/Downloads/Miniforge3-MacOSX-arm64.sh
```

Press **Return** to read the licence, type yes and press **Return**, accept the default location, and when it asks “Do you wish the installer to initialize Miniforge3 by running conda init?” type yes. Then **quit Terminal completely** (Command + Q) and open it again.

Checkpoint

Open a **new** terminal and type `conda --version`, then press Enter. You should see conda followed by a version number.

Careful

macOS may say an app is from an unidentified developer when you open it. You do not need to open any app in this guide, so you can ignore this.

Step 4. Download the NSDF-Data code

Your project will live in a folder called Research. Create it, go into it, and download the code. Type each line and press Enter:

```
mkdir -p ~/Research
cd ~/Research
git clone https://github.com/villano-lab/NSDF-Data.git
cd ~/Research/NSDF-Data
```

Downloading with `git clone` does not need a GitHub account. It makes a full copy of the project in a folder named NSDF-Data.

What these lines do

Line starts with	What it does
<code>mkdir</code>	Makes a new folder called Research.
<code>cd</code>	Moves you into a folder. <code>cd</code> stands for <i>change directory</i> , which is the same as opening a folder in a file window.
<code>git clone</code>	Downloads a copy of the project from the internet into a new folder called NSDF-Data.

Checkpoint

Type `ls` and press Enter. You are now inside the NSDF-Data folder (your prompt mentions it), so the list shows the project’s own folders and files, such as `python`, `R76`, `docs` and `README.md`.

Step 5. Create your analysis environment

An environment is a sealed box of programs with fixed versions. We make one for this project, called `darkmatter_cli_env`. Type these lines one at a time, pressing Enter after each. The last line

takes several minutes and prints a lot of text: wait for the prompt to come back.

```
conda create -n darkmatter_cli_env python=3.10 pip -y
conda activate darkmatter_cli_env
cd ~/Research/NSDF-Data
python -m pip install -r requirements.txt
```

What these lines do

Line	What it does
conda create	Makes a new, empty environment called <code>darkmatter_cli_env</code> , with Python 3.10 and the package installer <code>pip</code> inside it.
conda activate	Switches that environment on. While it is on, your prompt starts with <code>(darkmatter_cli_env)</code> .
cd	Moves you into the <code>NSDF-Data</code> folder, where the file <code>requirements.txt</code> is.
pip install -r requirements.txt	Installs every program on the project's list, at the exact versions in that file: the software that reads the detector data, numbers and plots, the test tool, and Jupyter, the notebook program you will use later.

Tip

`requirements.txt` is a plain text file in the project folder. It is the single list of what the project needs, so when a version changes, only that file changes. You can open it in any text editor to see the list.

Tip

Every time you open a new terminal, run `conda activate darkmatter_cli_env` first. When it is active, the start of each line shows `(darkmatter_cli_env)`.

Checkpoint

The last line of the output is a message, not an error, and the `(darkmatter_cli_env)` label is at the start of your prompt.

Step 6. Check that everything works

First, the tests. They use made-up data, so they need no download. Type each line and press Enter:

```
conda activate darkmatter_cli_env
cd ~/Research/NSDF-Data/python
python -m pytest -q
```

Checkpoint

The last line says 52 passed (it can be a larger number later, which is also fine). If it says failed, stop and ask the project lead, and do not change any files.

Then, the data tool:

```
nsdf-cli version
```

Checkpoint

It prints NSDF Dark Matter CLI: 0.5.0 (or a later version).

Finally, the notebook program:

```
jupyter lab --version
```

Checkpoint

It prints a version number, such as 4.4.0. If it says jupyter-lab is not found, check that your prompt starts with (darkmatter_cli_env), go to the NSDF-Data folder and run `python -m pip install -r requirements.txt` again.

What these commands do

Line	What it does
<code>conda activate</code>	Switches your environment on. Do this in every new terminal.
<code>cd</code>	Moves into the python folder of the project.
<code>python -m pytest -q</code>	Runs the project's built-in self-checks. The <code>-q</code> just means "keep the output short". 52 passed means every check worked.
<code>nsdf-cli version, jupyter lab --version</code>	Each one only asks a program to say which version it is. If it answers, it is installed.

Step 7. Let Jupyter notebooks use your environment

Jupyter is the notebook program you will use. Register your environment with it once:

```
python -m ipykernel install --user --name darkmatter_cli_env
```

Checkpoint

It prints a line that starts with `Installed kernelspec darkmatter_cli_env`.

Step 8. Get permission to save your work

You will not change the main project. You will save your work on your own **branch**, and for that the lead must give you access:

1. Make a free account at github.com if you do not have one.

2. Send your GitHub username to the project lead. They will add you to the project as a collaborator.
3. Once you have accepted the invitation, tell Git your name and email, so your work is labelled as yours. Use your own details:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

4. Create your personal branch (your own space to save work, separate from the main project). Use your own name, without spaces, in place of yourname:

```
cd ~/Research/NSDF-Data
git checkout -b student-yourname
```

Checkpoint

The terminal shows Switched to a new branch 'student-yourname'.

Rules for keeping your work safe

- **Never work on master or develop.** Those are the shared versions. Work only on your student- branch.
- **Never commit data.** The raw files in the idx folder are large and not part of the project. Download only from your home folder, never from inside NSDF-Data.
- **Do not run 07221203_2025_dump1_noise.ipynb** unless the project lead says it is fine. It overwrites a shared archive file.
- **Save often,** and commit your work when a piece is finished (see the first analysis guide).

You are done when

- ☐ Your terminal opens and `git --version` prints a version.
- ☐ `conda --version` prints a version, in a new terminal.
- ☐ `ls ~/Research` lists NSDF-Data.
- ☐ `python -m pytest -q` in the python folder ends with 52 passed.
- ☐ `nsdf-cli version` prints a version.
- ☐ `jupyter lab --version` prints a version.
- ☐ `jupyter kernelspec list` lists `darkmatter_cli_env`.
- ☐ `git branch` shows * student-yourname (after access is granted).

Next: *First analysis* (the guide for macOS).

If something goes wrong

1. Read the **last line** of the message. It usually says what is wrong.
2. Close the terminal, open a new one, and run `conda activate darkmatter_cli_env` again. Many problems are just the wrong environment.
3. Copy the command you ran and the message it printed, and send them to the project lead. A screenshot is also fine.

Where to get help

Anthony Villano, project lead: anthony.villano@ucdenver.edu

When you write, include the step number, the command you typed, and the last few lines the computer

printed. A screenshot helps. Nobody will be annoyed by a question about a step that did not work.

Session Info

Guide version 11, 9 October 2026, macOS edition. Written for beginners; please tell the project lead where a step was unclear, so the guide can be fixed.

Key links

- Notes site: <https://villano-lab.github.io/NSDF-Data/>
- Repository: <https://github.com/villano-lab/NSDF-Data>
- Miniforge downloads: <https://github.com/conda-forge/miniforge>
- Git downloads: <https://git-scm.com>